

Implement and compare the performance of a neural network using different activation functions (Sigmoid, ReLU, Tanh) on the MNIST dataset.

Aim:

- To implement neural networks using different activation functions
- To compare:
 - Training performance
 - Convergence speed
 - Classification accuracy

Theory – now need to write

◆ 1. MNIST Dataset

- Contains handwritten digit images (0–9)
- Each image: 28×28 pixels (784 features)
- Total classes: 10

◆ 2. Activation Functions

◆ Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- Output range: (0,1)
- Problem: Vanishing gradient

◆ Tanh

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- Output range: (-1,1)
- Better than sigmoid but still vanishing gradient

◆ ReLU

$$f(x) = \max(0, x)$$

- Fast convergence
- Most widely used

◆ 3. Comparison Concept

Function	Speed	Gradient Issue
Sigmoid	Slow	Yes
Tanh	Medium	Less
ReLU	Fast	No (mostly)

Algorithm

Step 1: Import Libraries

Step 2: Load Dataset

Step 3: Perform Preprocessing

Step 4: Model Function

Step 5: Train Models with Different Activations

Step 6: Compare Accuracy

Step 7: Plot Accuracy Curves

Step 8: Plot Loss Curves

Program

```
import numpy as np
import matplotlib.pyplot as plt
import time

from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input, Flatten
from tensorflow.keras.utils import to_categorical
(X_train, y_train), (X_test, y_test) = mnist.load_data()
# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

# One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

def create_model(activation):
    model = Sequential([
        Input(shape=(28,28)),
        Flatten(),
        Dense(128, activation=activation),
        Dense(64, activation=activation),
        Dense(10, activation='softmax')
    ])

    model.compile(optimizer='adam',
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])

    return model
activations = ['sigmoid', 'tanh', 'relu']
results = {}
histories = {}

for act in activations:
```

```

print(f"\nTraining with {act} activation")

model = create_model(act)

start = time.time()

history = model.fit(X_train, y_train,
                    epochs=5,
                    batch_size=128,
                    validation_split=0.2,
                    verbose=1)

end = time.time()

loss, acc = model.evaluate(X_test, y_test, verbose=0)

results[act] = {
    'accuracy': acc,
    'time': end - start
}

histories[act] = history

for act in results:
    print(f"{act} -> Accuracy: {results[act]['accuracy']:.4f}, Time: {results[act]['time']:.2f}s")

for act in activations:
    plt.plot(histories[act].history['val_accuracy'], label=act)

plt.title("Validation Accuracy Comparison")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()

for act in activations:

```

```
plt.plot(histories[act].history['val_loss'], label=act)

plt.title("Validation Loss Comparison")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()
```